
Sistemas Operacionais

Implementação de Processos

Contexto de Processo

Sistemas Operacionais Modernos são multitarefas

Chaveiam o processador(es) entre múltiplos programas

Mudança entre Processos demanda controle

Em que momento o processo parou?

Quais são os valores de suas variáveis?

Quais arquivos estão abertos?

Quais conexões de rede foram estabelecidas?

Contexto do Processo

Contador do Programa (PC)
Apontador da Pilha (SP)
Registradores usados

PCB (Process Control Block)

TCB (Task Control Block)

Informações relativas ao seu contexto e os demais dados necessários à sua gerência

<i>ID</i>	<i>Identificador da Tarefa</i>
<i>Estado</i>	<i>Nova, Pronta, Executando, Suspensa</i>
<i>Contexto do Processador</i>	<i>Dados nos registradores</i>
<i>Dados de Memória</i>	<i>Áreas da Memória usadas</i>
<i>Recursos em utilização</i>	<i>Arquivos, portas, dentre outras</i>
<i>Dados de gerenciamento</i>	<i>Prioridade, proprietário, data de início, dentre outras</i>

Kernel tem diferentes listas, especialmente a de prontas para executar

Troca de Contexto

Ação fundamental para Sistemas Multitarefa

Trocar transparentemente tarefas em execução (suspender uma, reativar outra)

Requer

Salvar contexto de uma tarefa em seu TCB

Escolher outra tarefa para execução

Restaurar tal contexto em momento futuro no processador

Motivação:

Aproveitar ciclos chamados CPU e I/O Burst



Troca de Contexto

Dois componentes importantes:

Escalonador

Qual tarefa deve receber o processador quando outra é suspensa?

Dispatcher

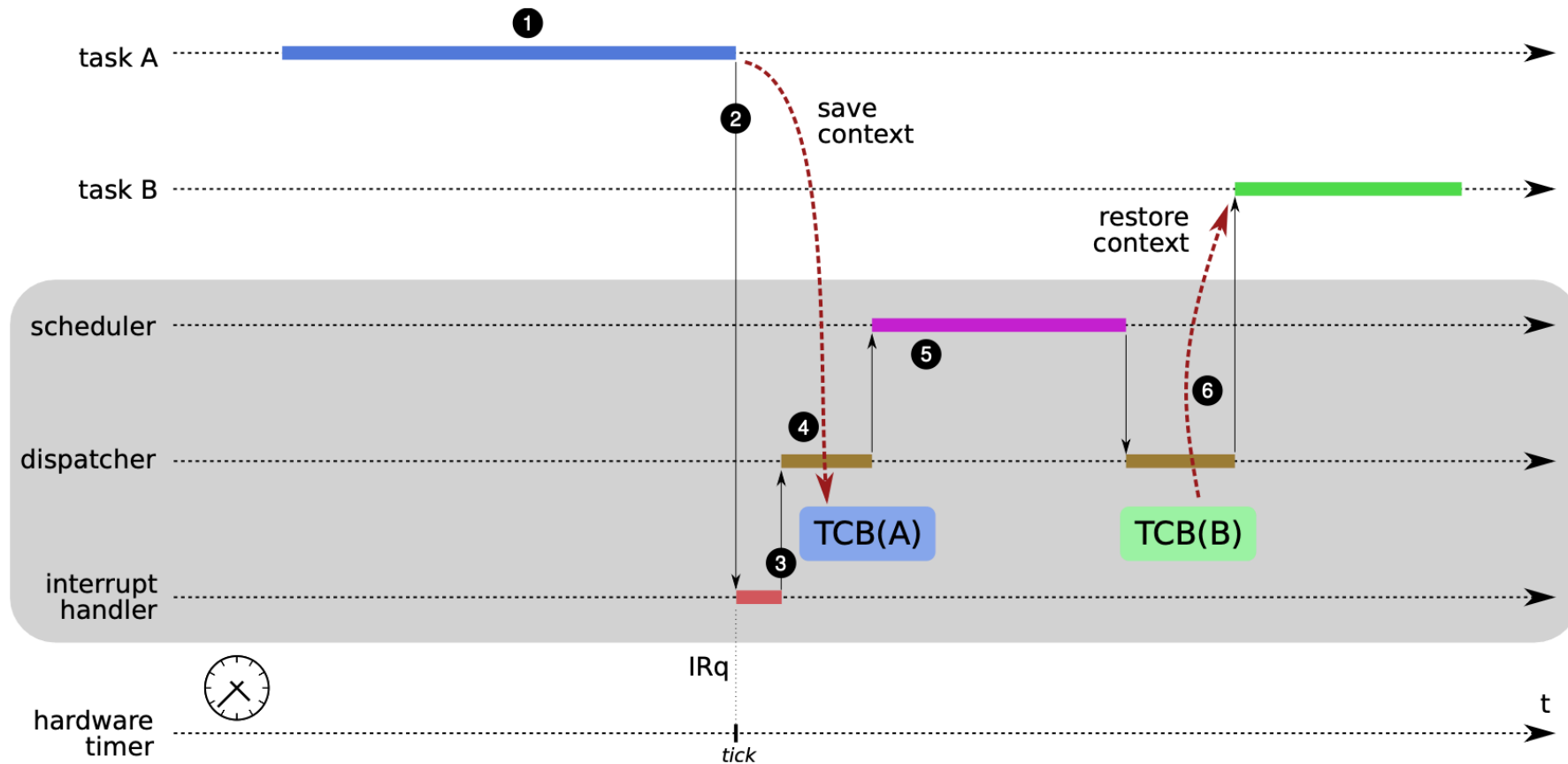
Dar controle do processador à tarefa escolhida pelo escalonado

Dispatcher Latency Time



- 1. Troca de Contexto*
- 2. Efetiva o modo usuário*
- 3. Resume a execução do processo selecionado*

Troca de Contexto

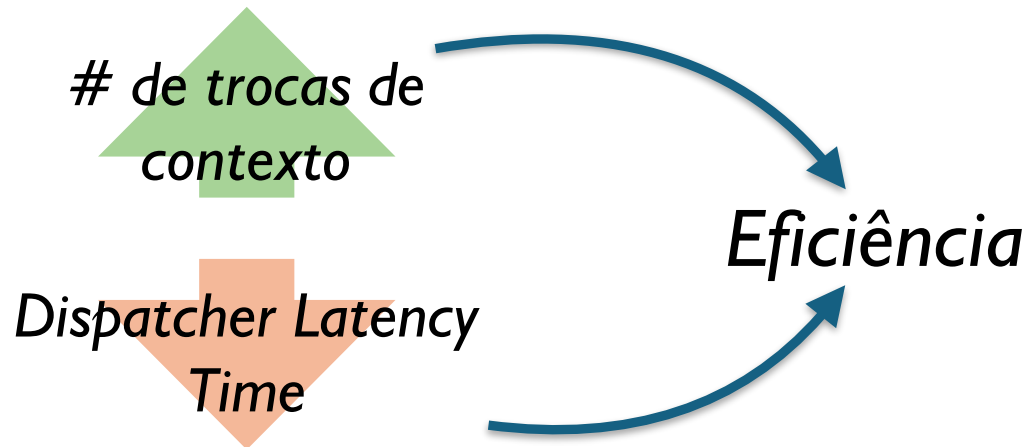


Troca de Contexto

Possíveis causas para troca de contexto



1. Término do quantum
2. Evento de periférico
3. System Call
4. Erro de Execução



$$\varepsilon = \frac{t_q}{t_q + t_{tc}}$$

t_q = quantum

t_{tc} = Dispatcher Latency Time

Troca de Contexto

Se $t_{tc} = 100\mu s$ e $t_q = 10 ms$

$$\epsilon = \frac{0.01}{0.01 + 0.0001}$$

$$\epsilon = 0.9900990099$$

99%

Se $t_{tc} = 1 ms$ e $t_q = 10 ms$

$$\epsilon = \frac{0.01}{0.01 + 0.001}$$

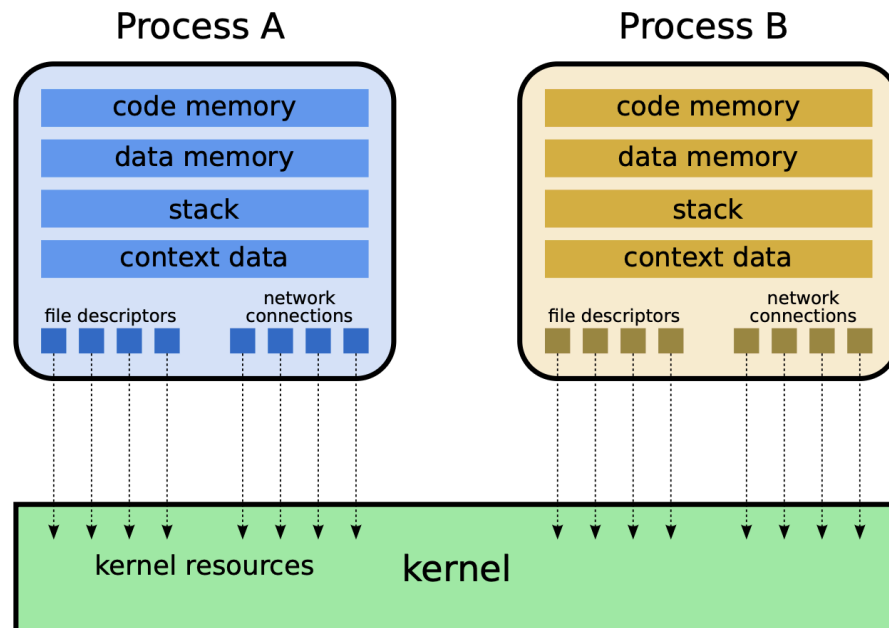
$$\epsilon = 0.90909090$$

91%

Processo

Capsula isolada de execução

Tarefa e recursos



Capsula isolada de execução

Tarefa e recursos

*Processos possuem uma
linha sequencial de execução*

Modificação por Threads

Kernell

PCBs

PID (Process Identifier)

Gestão de Processos

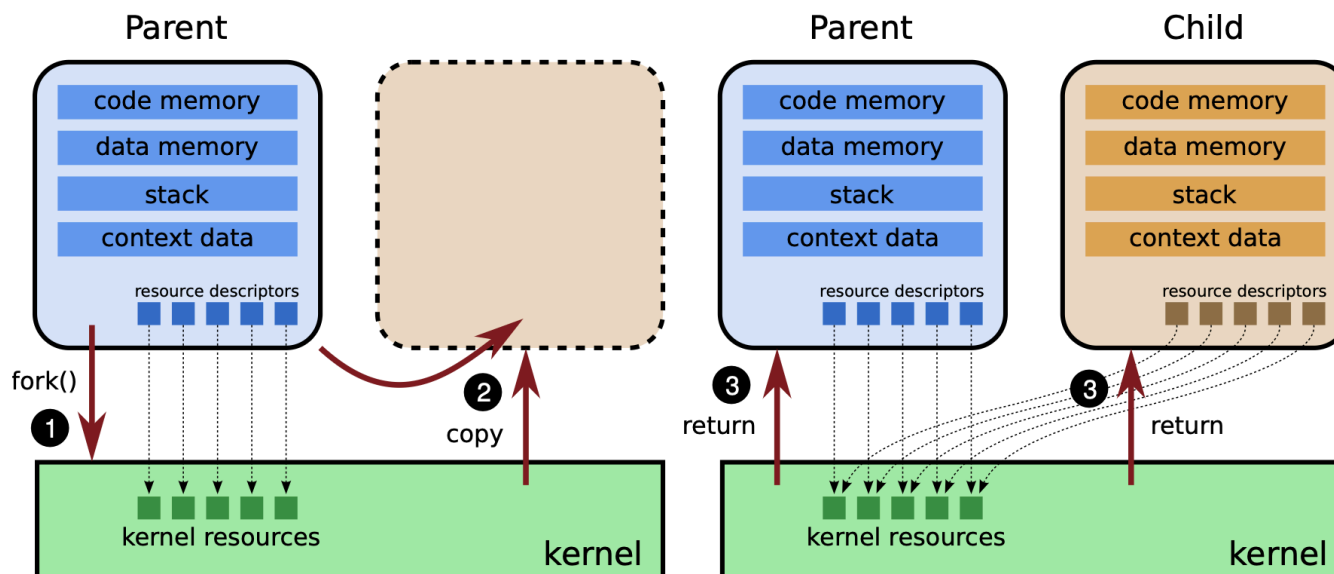
Ações básicos do SO: criar e encerrar processos

Cada SO tem suas próprias chamadas

<i>Ação</i>	<i>Windows</i>	<i>Linux</i>
<i>Criar um novo processo</i>	CreateProcess()	fork(), execve()
<i>Encerrar processo corrente</i>	ExitProcess()	exit()
<i>Encerrar outro processo</i>	TerminateProcess()	kill()
<i>Obter o PID do processo corrente</i>	GetCurrentProcessId()	getpid()

Criação via fork/execve

Processo origem (pai) cria uma réplica (filho) de si mesmo



*Separação entre
criar e carregar
programas*

Simplicidade

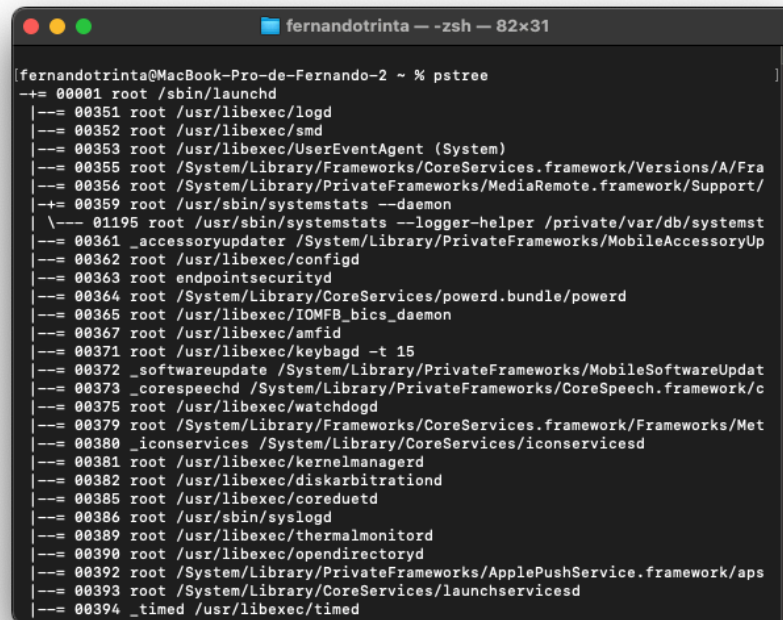
*Reutilização de
mecanismos existentes*

*Flexibilidade para o
processo pai*

Hierarquia de Processos

Processos são filhos de outros processos

Gerenciamento coletivo de processos de uma aplicação ou sessão



```
fernandotrinta@MacBook-Pro-de-Fernando-2 ~ % ps tree
--= 00001 root /sbin/launchd
|--= 00351 root /usr/libexec/logd
|--= 00352 root /usr/libexec/smd
|--= 00353 root /usr/libexec/UserEventAgent (System)
|--= 00355 root /System/Library/Frameworks/CoreServices.framework/Versions/A/Fra
|--= 00356 root /System/Library/PrivateFrameworks/MediaRemote.framework/Support/
--+= 00359 root /usr/sbin/systemstats --daemon
\--= 01195 root /usr/sbin/systemstats --logger-helper /private/var/db/systemst
--= 00361 _accessoryupdater /System/Library/PrivateFrameworks/MobileAccessoryUp
--= 00362 root /usr/libexec/configd
--= 00363 root endpointsecurityd
--= 00364 root /System/Library/CoreServices/powerd.bundle/powerd
--= 00365 root /usr/libexec/IOMFB_bics_daemon
--= 00367 root /usr/libexec/amfid
--= 00371 root /usr/libexec/keybagd -t 15
--= 00372 _softwareupdate /System/Library/PrivateFrameworks/MobileSoftwareUpdat
--= 00373 _corespeechd /System/Library/PrivateFrameworks/CoreSpeech.framework/c
--= 00375 root /usr/libexec/watchdogd
--= 00379 root /System/Library/Frameworks/CoreServices.framework/Frameworks/Met
--= 00380 _iconservices /System/Library/CoreServices/iconservicesd
--= 00381 root /usr/libexec/kernelmanagerd
--= 00382 root /usr/libexec/diskarbitrationd
--= 00385 root /usr/libexec/coreduetd
--= 00386 root /usr/sbin/syslogd
--= 00389 root /usr/libexec/thermalmonitord
--= 00390 root /usr/libexec/opendirectoryd
--= 00392 root /System/Library/PrivateFrameworks/ApplePushService.framework/aps
--= 00393 root /System/Library/CoreServices/launchservicesd
--= 00394 _timed /usr/libexec/timed
```

Encerramento de Processos

Gracefully

Saída normal e programática

`exit()`

Libera recursos de volta ao SO

Abrupta

Erros

Sinalização do SO

Sinalização de outro processo

Processos Zumbis/Orfãos

Processo Zumbi:

Processo finalizado, mas ainda com entrada na lista de processos ativos

Processo filho termina, mas processo pai não consegue ser informado

Excesso de zumbis pode impedir inclusão de novos processos

Processo Orfão:

Processo pai encerra antes de seus filhos

Solução: Realocação para novo processo

UNIX: processo init (primeiro a ser carregado no boot do SO)

Threads

Lightweight process (processo leve)

Fluxos de execução independente, dentro de um mesmo processo

Thread Local Storage

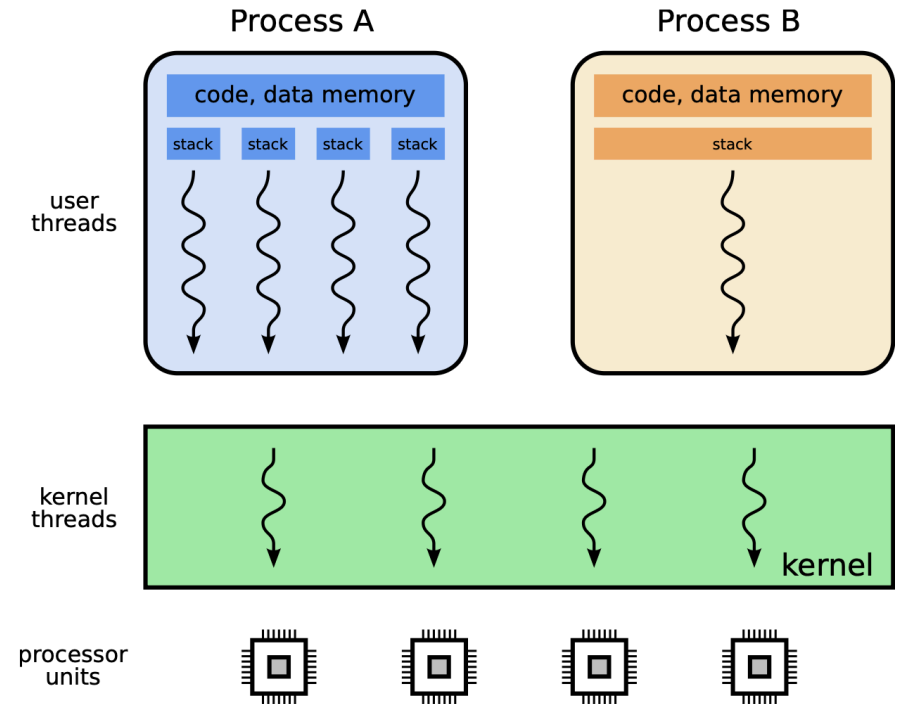
Código de Execução

Contexto Local

Separação Importante:

Threads de Processos do Usuário (User-Level Threads)

Threads de rotinas do núcleo (Kernel-Level Threads)



Modelos de Threads

*Como é feito o mapeamento
entre Threads do Usuário e
Threads no Kernel?*



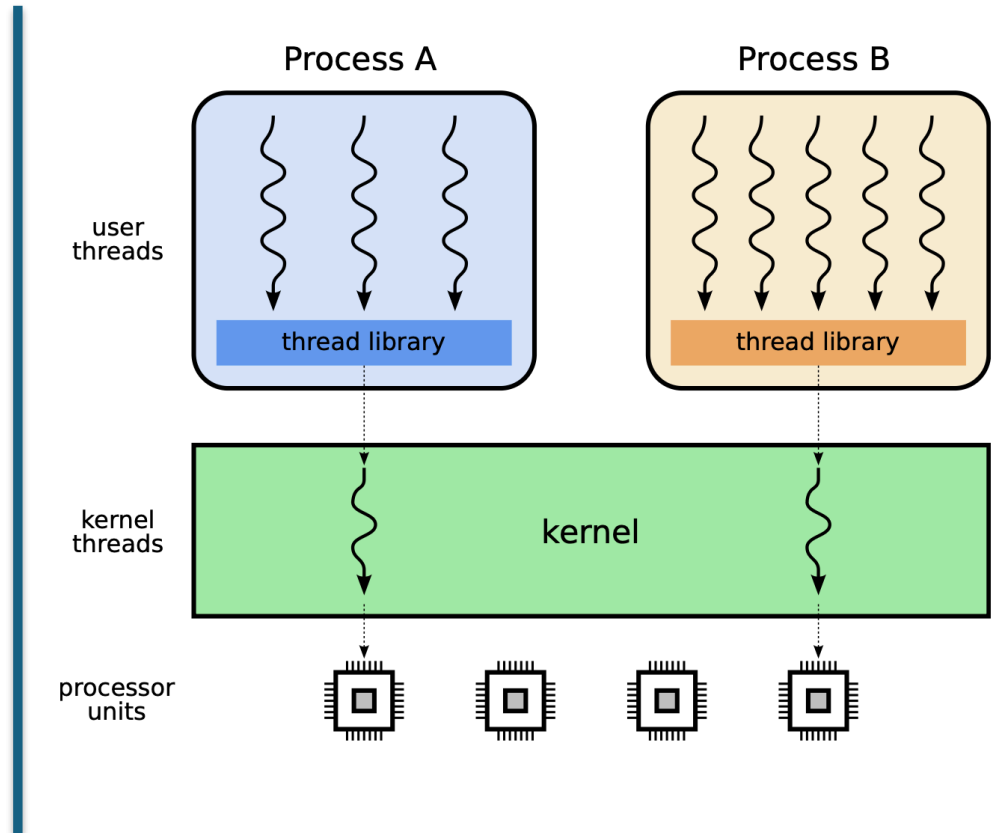
Modelo N:1

*Usado em SOs mais antigos
a.k.a. Green Threads (fibers)
Várias User-Level Threads
mapeadas em uma única Kernel
Level Thread*

Implementação Simplificada 👍

*Possível Gargalo para aplicações com
muitas threads 🙇*

Sem paralelismo interno 🙇



Modelo 1:1

Gestão do paralelismo pelo próprio kernel

Paralelismo real

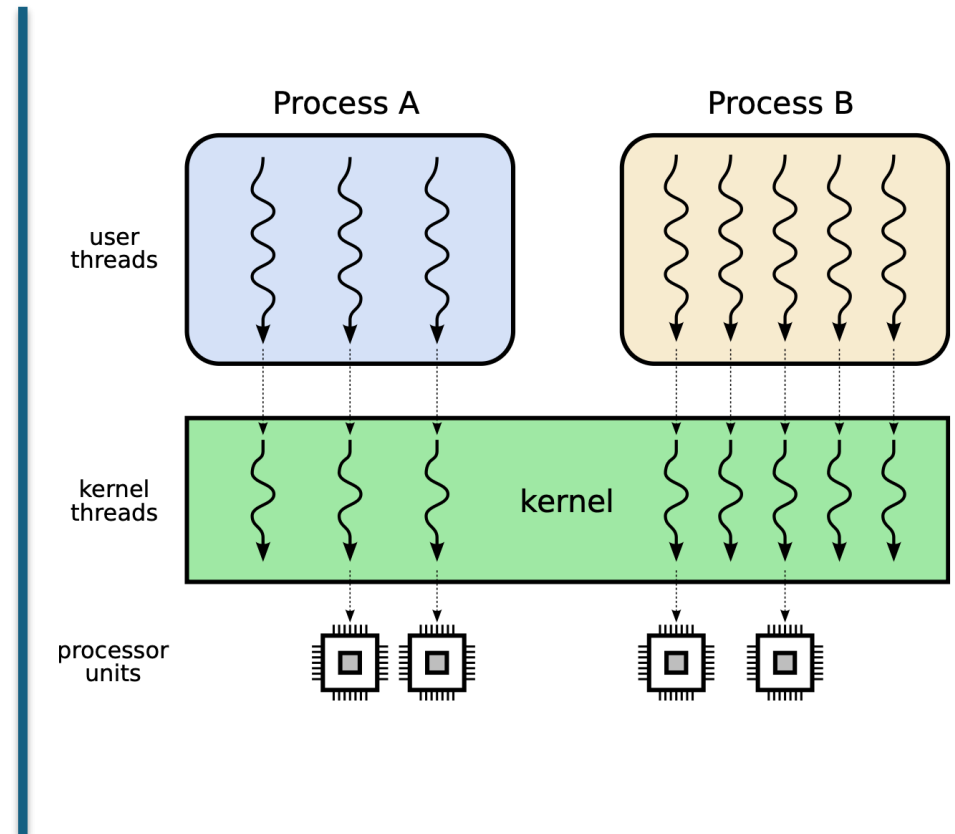
Características

Evita gargalos e bloqueios entre Threads 👍

Distribuição mais justa entre processos 👍

Custo elevado para o sistema 👎

Windows NT e UNIX



Modelo N:M

Modelo Híbrido entre os anteriores

Biblioteca gerencia um conjunto de N threads de usuário (dentro do processo), que é mapeado em $M < N$ threads no núcleo

Thread Pool

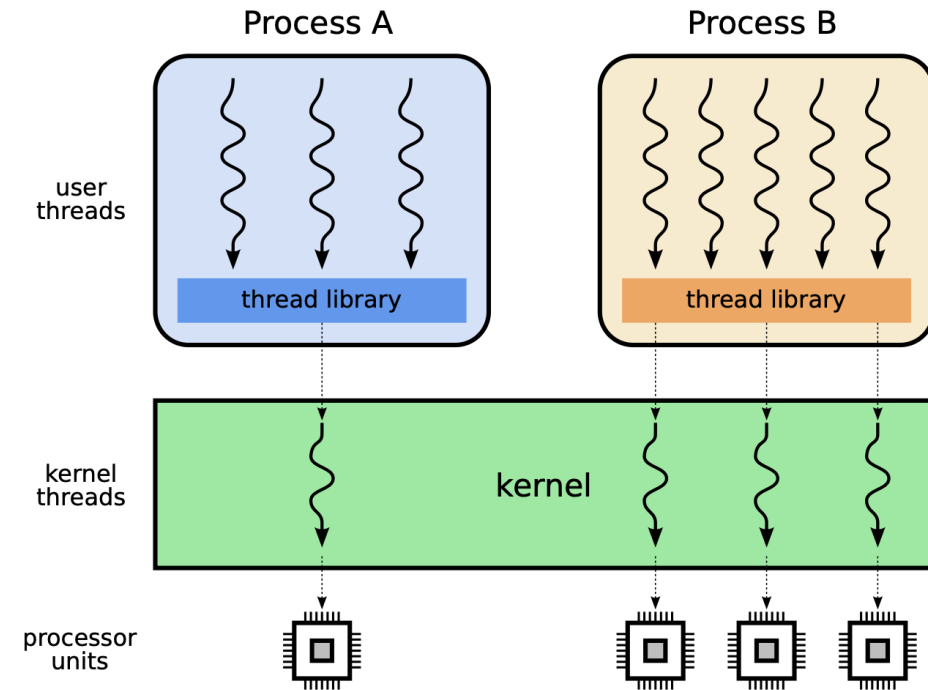
Características

Alta interatividade (1:1) e Escalabilidade (N:1) 👍

Complexidade de Implementação 👎

Gestão das Kernel-Level Threads 👎

Solaris



Comparação

Modelo	N:1	1:1	N:M
Resumo	<i>N threads do processo mapeados em uma thread de núcleo</i>	<i>Cada thread do processo mapeado em uma thread de núcleo</i>	<i>N threads do processo mapeados em $M < N$ threads de núcleo</i>
Implementação	<i>no processo (biblioteca)</i>	<i>no núcleo</i>	<i>em ambos</i>
Complexidade	<i>Baixa</i>	<i>Média</i>	<i>Alta</i>
Custo de Gerência	<i>Baixo</i>	<i>Médio</i>	<i>Alto</i>
Escalabilidade	<i>Alta*</i>	<i>Baixa</i>	<i>Alta</i>
Paralelismo no mesmo processo	<i>Não</i>	<i>Sim</i>	<i>Sim</i>
Troca de contexto entre Threads no mesmo processo	<i>Rápida</i>	<i>Lenta</i>	<i>Rápida</i>
Divisão de recursos entre tarefas	<i>Injusta</i>	<i>Justa</i>	<i>Variável (mapeamento entre ULT e KLT é dinâmico)</i>
Exemplos	<i>GNU Portable Threads, Microsoft UMS</i>	<i>Windows, Linux</i>	<i>Solaris, FreeBSD KSE</i>

Thread libraries

Toolkit para facilitar a programação com Threads

Busca por padronização

Funções Básicas:

Create - Criar novas Threads

Terminate - Encerrar Threads

Join - Esperar que uma Thread termine

Uso de Mutex ou sync (acesso a regiões críticas)

Mais populares:

POSIX (PThreads), Windows Threads e Java Threads

Dois possíveis níveis:

User Level:

Fácil para criar Threads

Sem paralelismo real

Kernel Level

Criação mais lenta

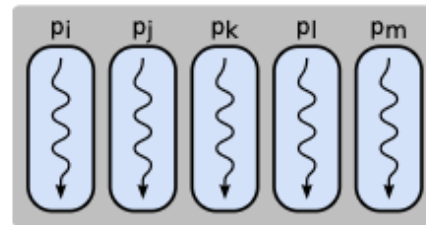
Paralelismo puro

Processos X Threads

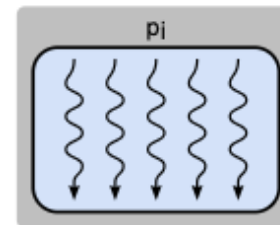
Em sistemas complexos, que envolvem várias tarefas dependentes sendo executadas ao mesmo tempo, qual forma de implementação é mais adequada?



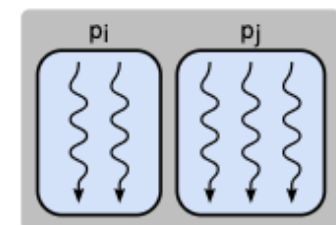
Processos X Threads



com processos



com threads



híbrido

Custo de criação	<i>alto</i>	<i>Baixo</i>	<i>Médio</i>
Troca de Contexto	<i>Lenta</i>	<i>Rápida</i>	<i>Variável</i>
Uso de memória	<i>alto</i>	<i>Baixo</i>	<i>Médio</i>
Compartilhamento de dados	<i>Canais de comunicação e memória compartilhada</i>	<i>Variáveis Globais e dinâmicas</i>	<i>Ambos</i>
Robustez	<i>Alto (erros autocontidos)</i>	<i>Baixo (erro afeta todas)</i>	<i>Médio, (afeta threads no mesmo processo)</i>
Segurança	<i>Alta (usuários e permissões distintas)</i>	<i>Baixa (permissões herdadas do processo onde executam)</i>	<i>Alta (agrupamento de threads com mesma permissão)</i>
Exemplos	<i>Apache WebServer, Postgres</i>	<i>Apache WebServer v2, MySQL</i>	<i>Chrome, Firefox e Oracle DB</i>